AI-ACCELERATED SOFTWARE DEVELOPMENT

Applied Practice

Guided artifact delivery using AI-assisted workflows in sandbox and approved enterprise repositories.

4 weeks · 4 workstreams · 8–10 hours per week

Confidential. Prepared exclusively for Ensemble Health Partners.

The bridge between Foundations and real-project coaching

4	4	8–10	2h
Weeks of Applied Work	Core Workstreams	Hours Per Week	Guidance Content / Week

- Participants stop working at the concept level and begin producing reviewable engineering artifacts
- Work happens inside sandbox or approved enterprise repositories — no unapproved external SaaS
- Outputs are judged on engineering quality, not prompt cleverness
- Participants build a usable portfolio: architecture docs, test suites, security artifacts, runbooks

Not abstract exposure to AI tools.
Repeatable delivery under enterprise constraints — the same conditions real engineering teams operate under.

Four delivery characteristics that distinguish applied work from a lab

ENTERPRISE-SAFE

No unapproved workflows or casual data handling

- No unapproved external SaaS
- No casual code upload patterns
- Approved models and bounded execution only
- Work inside sandbox or approved repos

REAL DELIVERABLES

Every week produces reviewable artifacts

- Artifacts reviewed each week
- Quality judged, not prompt cleverness
- Portfolio of process and code artifacts
- Outputs usable in real delivery

PRODUCTION-ORIENTED

Mirrors enterprise software team conditions

- Exercises mirror enterprise team reality
- Maintainability and reviewability required
- Operational readiness emphasized
- Real team delivery, not isolated labs

COACH-SUPPORTED

Guidance on method and quality, not code

- Off-network or approved review channels
- Coaches guide method, not execution
- Participants own execution in their env
- No sensitive code handled by coaches

→ This is not a second pass through Foundations. It is delivery practice under the same constraints that govern production engineering.

Prerequisites — what participants must already be able to do

Completion of Foundations required

Foundations is the safety and operating-model gate. It is not a suggested prerequisite — it is the precondition for safe participation in applied work.

- ✓ Work in the Claude Code CLI confidently without setup guidance
- ✓ Scope context intentionally — choose the minimum working set
- ✓ Produce reviewed plans in Plan Mode before any mutation
- ✓ Treat AI output as untrusted until verified by tests or review
- ✓ Narrate changes with clear risk and testing notes

What Applied Practice adds

From posture to output

Foundations instills the operating model. Applied Practice turns that model into concrete, reviewable artifacts inside real repositories.

From exercises to deliverables

Exercises disappear. Deliverables remain. Every week ends with an artifact that can be reviewed, inherited, and defended to engineering or security stakeholders.

From individual to team-scale

Cohort review, coaching, and artifact submission mirror the review loops of real engineering teams — not solo learning environments.

01 Codebase Analysis and Documentation

`Week 3` `~4 hours` `Read-only first`

You'll learn

- Safe repo inspection with read-only first-pass discipline
- Dependency and boundary mapping across system components
- Hotspot identification using churn, complexity, and low-test areas
- Documentation that is actionable for new engineers — not generic summary text

> Inspect a production-style repository safely. Identify system boundaries. Create a risk-ranked backlog of safe improvements.

Deliverables

- Architecture README covering purpose, runtime model, risks, and key boundaries
- `ADR v1` documenting one significant decision with trade-offs
- Dependency and boundary map notes
- Risk-ranked backlog with the top safe PR candidates
- One selected safe change with plan and acceptance criteria

02 Testing and Coverage Acceleration

Week 4 | ~4 hours | Unit tests · Deterministic patterns only

You'll learn

- Baseline coverage measurement and interpretation before adding tests
- High-ROI target selection based on risk and change frequency
- Human review loops for AI-generated tests — no blind trust
- Coverage delta reporting that communicates value clearly to stakeholders

> Use AI where it improves confidence — not where it creates unmaintainable test debt. Readable tests survive future maintenance; generated noise does not.

Deliverables

- Coverage baseline before any changes are made
- Test plan defining what to test and why — not just what AI generated
- New tests with edge cases and readable, maintainable naming
- Coverage delta report after changes with clear before/after framing
- Refactoring notes where testability required code improvements

03 Security and Compliance

Week 5 · ~4 hours · Security review discipline · Compliance-ready evidence

You'll learn

- Prompt redaction and safe-data handling patterns for enterprise workflows
- Prompt injection attack and defense in practical delivery scenarios
- Permission-boundary reasoning for agentic execution environments
- Minimum viable compliance documentation for AI-assisted work
- Converting abstract policy into reviewable execution controls

Deliverables

- Prompt policy or working policy addendum for the chosen workflow
- Redaction checklist tied to actual delivery scenarios
- Prompt-review workflow showing who checks what before submission
- Injection drill findings with concrete, reviewable mitigations
- Permissions and data-handling notes for the repo or workflow
- Compliance rationale — why the workflow is acceptable, or what blocks it

04 Observability

Week 6 | ~4 hours | Operational readiness · Reconstructable system behavior

You'll learn

- Structured logging with defined fields and stable, documented schemas
- Correlation or trace ID propagation across requests and components
- Minimum viable operational instrumentation that actually helps on-call
- Runbook authoring for local run, failure triage, and log interpretation
- How observability reduces ambiguity during AI-assisted delivery and later debugging

Deliverables

- Structured logging implementation or logging retrofit with defined fields
- Correlation ID propagation across the relevant request path
- Log field reference or schema note for the team
- Operational runbook covering startup, normal flow, and failure diagnosis
- Troubleshooting examples tied to actual log output
- Observability gap note — what still is not instrumented well enough

Four weeks · one workstream per week · every week ends with reviewable artifacts

Week 3

WORKSTREAM 01 — ANALYSIS

- Inspect production-style repo safely — read-only first
- Produce architecture notes and an ADR
- Create safe-change backlog with explicit reasoning
- Select one safe PR candidate with acceptance criteria

~4 hours applied work

Week 4

WORKSTREAM 02 — TESTING

- Measure coverage baseline before touching anything
- Choose highest-value test targets by risk and churn
- Generate and review maintainable, readable tests
- Report coverage delta with clear before/after framing

~4 hours applied work

Week 5

WORKSTREAM 03 — SECURITY

- Practice prompt redaction and workflow review
- Run an injection-defense drill with findings
- Produce evidence the workflow survives security review
- Document permissions and data-handling boundaries

~4 hours applied work

Week 6

WORKSTREAM 04 — OBSERVABILITY

- Add structured logging and correlation propagation
- Produce an operational runbook with real log examples
- Document instrumentation gaps honestly
- Make the artifact easier to diagnose and hand off

~4 hours applied work

→ Each week: **2 hours** guidance content + **5–6 hours** applied artifact work + **1–2 hours** review, submission, and cohort discussion.

What participants build across four weeks

Code Artifacts

- Targeted test suites
- Structured logging improvements
- Safe bounded implementation changes
- Low-risk PR candidates

Documentation

- Architecture READMEs
- Architecture Decision Records
- Runbooks
- Dependency and boundary notes

Process Artifacts

- PR narrations
- Plan-mode outputs with acceptance criteria
- Coverage delta reports
- Risk-ranked backlogs

Security and Compliance Artifacts

- Prompt policy notes
- Redaction checklists
- Injection drill findings
- Permissions and data-handling notes
- Workflow acceptability rationale

Observability Artifacts

- Logging field definitions
- Correlation propagation notes
- Troubleshooting guides
- Instrumentation gap assessments

> All artifacts must be reviewable by engineering, security, and delivery stakeholders — not just functional for the participant.

Assessment rubrics — what good looks like

PR Rubric

- ✓ Clear statement of what changed
- ✓ Clear explanation of why it changed
- ✓ Risk assessment included
- ✓ Testing approach documented
- ✓ Follow-up items identified

Documentation Rubric

- ✓ Actionable for another engineer, not just the author
- ✓ Accurate to current system state
- ✓ Risks clearly identified — not hidden or minimized
- ✓ Run and build steps verified where relevant
- ✓ Trade-offs explained rather than concealed

Tests Rubric

- ✓ Readable and well named — purpose is obvious
- ✓ Deterministic and non-flaky
- ✓ Edge cases included where appropriate
- ✓ Maintainable structure that survives future changes
- ✓ Meaningful confidence improvement shown

Security Rubric

- ✓ Sensitive-data boundaries are explicit
- ✓ No unsafe prompt content normalized
- ✓ Mitigation steps are concrete and reviewable
- ✓ Approvals and ownership are visible
- ✓ Evidence is sufficient for later audit

Observability rubric: Logs structured and purposeful · Correlation usable end-to-end · Runbook steps practical · Troubleshooting maps to real signals · Blind spots acknowledged explicitly

By the end of Applied Practice, participants can do

- ✓ Inspect an enterprise codebase safely and extract usable architectural understanding
- ✓ Demonstrate secure and compliant AI-assisted workflow behavior with evidence
- ✓ Submit artifacts reviewable by engineering, security, and delivery stakeholders
- ✓ Improve confidence through targeted testing — not indiscriminate AI generation
- ✓ Make system behavior more observable and easier to support and hand off
- ✓ Operate at a level that mirrors real team delivery — not isolated training exercises

What comes next

Embedded Project Coaching or Applied Delivery on Real Repositories — real repo outcomes, team-specific rollout decisions, and measurable operational impact at production scale.

APPLIED PRACTICE

Artifacts over exercises.
Delivery over exposure.

Applied Practice is where Foundations habits become engineering outputs.
Every week ends with something a real team can review, inherit, and build on.